

NumPy & Pandas

Complete Core Concept Guide

Hinglish explanation | Real-world use cases | Interview Q&A

Data Science Interview Prep — 2026 Edition

📄 Table of Contents

1. NumPy Kya Hai Aur Kyu Zaruri Hai
2. NumPy Array Banane Ke Tarike
3. Array Attributes (shape, dtype, ndim, size)
4. Indexing, Slicing & Fancy Indexing
5. Boolean Indexing / Filtering
6. Broadcasting
7. Reshaping & Flattening
8. Stacking & Splitting
9. Aggregation Functions & Axis Concept
10. Vectorization vs Loops
11. Linear Algebra Operations
12. Random Module
13. Copy vs View
14. Pandas Kya Hai Aur Kyu Zaruri Hai
15. Series & DataFrame Basics
16. Data Reading/Writing (CSV, Excel, JSON)
17. loc vs iloc vs at/iat
18. Filtering & Conditional Selection
19. Missing Data Handling
20. GroupBy Deep Dive
21. Merge, Join & Concat
22. Pivot Table & Crosstab
23. Apply, Map & Applymap
24. Sorting & Ranking
25. Duplicate Handling
26. DateTime Handling
27. Frequently Asked Interview Questions (NumPy)
28. Frequently Asked Interview Questions (Pandas)

Part 1: NumPy Deep Dive

1. NumPy Kya Hai Aur Kyu Zaruri Hai

ENGLISH

NumPy (Numerical Python) is a library used for **fast numerical computation**. Regular Python lists are slow for large calculations because they store generic objects. NumPy's `ndarray` (n-dimensional array) stores data in a contiguous block of memory, making operations extremely fast (C-level speed).

HINGLISH

NumPy (Numerical Python) ek library hai jo **fast numerical computation** ke liye use hoti hai. Python ke normal lists slow hote hain bade calculations ke liye kyunki wo generic objects store karte hain. NumPy ka `ndarray` (n-dimensional array) memory mein contiguous block mein data store karta hai, jisse operations bahut fast (C-level speed) hote hain.

Use case: Machine Learning models (scikit-learn, TensorFlow) ke andar saara data NumPy arrays ke form mein hi process hota hai. Image processing mein bhi image ko pixel array (height x width x channels) ke roop mein NumPy array bana kar handle karte hain.

```
import numpy as np
a = [1, 2, 3, 4] # normal python list - slow
b = np.array([1, 2, 3, 4]) # numpy array - fast, vectorized
```

2. NumPy Array Banane Ke Tarike

ENGLISH

There are multiple functions to create arrays, each suited to a different situation.

HINGLISH

Array banane ke multiple functions hain, har ek ka apna use case hai.

```
np.array([1,2,3]) # list se array
np.zeros((2,3)) # sab zero, shape (2,3)
np.ones((3,3)) # sab one
np.full((2,2), 7) # ek fix value se fill
np.arange(0,10,2) # range: 0,2,4,6,8
np.linspace(0,1,5) # 0 se 1 ke beech 5 equally spaced values
np.eye(3) # identity matrix
np.random.rand(2,2) # random values 0-1 ke beech
```

Use case: `np.zeros()` ka use aksar ek empty matrix initialize karne ke liye hota hai (jaise neural network weights initialize karna), aur `np.linspace()` ka use graph plotting mein x-axis values generate karne ke liye hota hai.

3. Array Attributes

ENGLISH

Every array has a few attributes that describe its structure — these are accessed directly as properties, without calling a function.

HINGLISH

Har array ke kuch attributes hote hain jo uski structure batate hain — bina kisi function call ke, seedhe property ki tarah access hote hain.

Attribute	Kya batata hai	Example
<code>.shape</code>	Dimensions (rows, cols)	<code>(3,4)</code>
<code>.ndim</code>	Kitne dimensions hain	<code>2</code>
<code>.size</code>	Total elements	<code>12</code>
<code>.dtype</code>	Data type (int64, float64)	<code>int64</code>
<code>.itemsize</code>	Ek element kitne bytes leta hai	<code>8</code>

Use case: Deep learning mein tensor ka shape check karna sabse common debugging step hai — agar shape match nahi karta to matrix multiplication error aata hai.

4. Indexing, Slicing & Fancy Indexing

ENGLISH

Indexing/slicing is used to extract specific elements or sub-arrays. Fancy indexing means passing a list/array of indices to select multiple elements at once.

HINGLISH

Array ke specific elements ya sub-arrays nikalne ke liye indexing/slicing use hoti hai. Fancy indexing matlab ek list/array ke indices pass karke multiple elements ek saath select karna.

```
arr = np.array([10,20,30,40,50])
arr[1]      # 20
arr[1:4]    # [20,30,40]
arr[::-1]   # reverse
arr[[0,2,4]] # fancy indexing -> [10,30,50]
```

```
mat = np.array([[1,2,3],[4,5,6]])
mat[0,1]      # 2 (row 0, col 1)
mat[:,1]      # pura column 1 -> [2,5]
```

Use case: Dataset se specific rows/columns nikalne ke liye (jaise sirf training data ke first 100 samples lena) fancy indexing kaam aati hai.

5. Boolean Indexing / Filtering

ENGLISH

By applying a condition, you can filter out only those elements from the array that satisfy the condition — without writing any loop.

HINGLISH

Condition laga kar array se sirf wo elements filter kar sakte hain jo condition satisfy karte hain — bina kisi loop ke.

```
arr = np.array([5,12,18,3,25])
mask = arr > 10      # [False,True,True,False,True]
arr[mask]            # [12,18,25]
arr[arr > 10] = 0    # condition true wahan value replace
```

Use case: Outliers remove karna (jaise salary dataset mein negative values ya bahut zyada extreme values filter karna) is technique se ek line mein ho jata hai.

6. Broadcasting

ENGLISH

Broadcasting is a rule that lets NumPy perform operations between arrays of different shapes without writing an explicit loop. The smaller array is automatically "stretched" to match the shape of the larger one.

HINGLISH

Broadcasting ek rule hai jisse NumPy different shapes ke arrays ke beech operation allow karta hai, bina explicitly loop likhe. Chhoti array automatically "stretch" ho jati hai bade array ke shape ke according.

```
a = np.array([1,2,3])
b = 2
a * b          # [2,4,6] - scalar broadcast hua

mat = np.array([[1,2,3],[4,5,6]])
row = np.array([10,20,30])
mat + row      # har row mein row add ho jayega
```

Use case: Image processing mein har pixel se ek fix brightness value add/subtract karna (jaise image ko bright/dark karna) broadcasting se ek line mein hota hai.

7. Reshaping & Flattening

```
arr = np.arange(12)
arr.reshape(3,4)  # 1D ko 2D banaya (3 rows, 4 cols)
arr.reshape(-1,1) # -1 matlab "isko tum khud calculate karo"
arr.flatten()     # multi-D ko wapas 1D (copy return karta hai)
arr.ravel()       # flatten jaisa, lekin view return karta hai (fast)
```

Use case: Image data (height, width, channels) ko ML model mein feed karne se pehle usko flatten/reshape karna padta hai kyunki model 1D ya

specific shape expect karta hai.

8. Stacking & Splitting

```
a = np.array([1,2,3])
b = np.array([4,5,6])
np.vstack((a,b))    # vertically stack -> 2 rows
np.hstack((a,b))    # horizontally stack -> 1 row, 6 cols
np.concatenate([a,b])

np.split(arr, 3)     # array ko 3 barabar parts mein todna
```

Use case: Multiple batches ka data combine karna (jaise alag alag CSV se load kiye gaye arrays ko ek saath jodna) ya train/test split ke liye data ko divide karna.

9. Aggregation Functions & Axis Concept

ENGLISH

The **axis** concept confuses beginners the most. **axis=0** means the operation runs column-wise (going down through the rows), and **axis=1** means it runs row-wise (going across through the columns).

HINGLISH

Axis concept sabse zyada confuse karta hai beginners ko. **axis=0** matlab operation column-wise hoga (rows ke through jayega), aur **axis=1** matlab row-wise (columns ke through jayega).

```
mat = np.array([[1,2,3],[4,5,6]])
mat.sum()          # total sum -> 21
mat.sum(axis=0)    # column-wise sum -> [5,7,9]
mat.sum(axis=1)    # row-wise sum -> [6,15]
mat.mean(), mat.std(), mat.min(), mat.max()
mat.argmax()       # max value ka index
```

Use case: Har student ke total marks (row-wise sum) ya har subject ka average (column-wise mean) nikalne mein axis concept directly use hota hai.

10. Vectorization vs Loops

ENGLISH

Vectorization means using direct array operations instead of loops — NumPy runs these operations internally using C code, which is 10-100x faster than Python loops.

HINGLISH

Vectorization matlab loop ki jagah direct array operations use karna — NumPy internally C code se ye operations run karta hai jo Python loops se 10-100x tak fast hote hain.

```
# Slow way (loop)
result = []
for x in arr:
    result.append(x*2)

# Fast way (vectorized)
result = arr * 2
```

Use case: Bade datasets (lakhs rows) par calculation karte waqt loop use karne se program bahut slow ho jata hai; vectorization se same kaam seconds mein hota hai.

11. Linear Algebra Operations

```
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])
A.T          # transpose
A @ B        # matrix multiplication (dot product)
np.linalg.inv(A)  # inverse matrix
np.linalg.det(A)  # determinant
np.linalg.eig(A)  # eigenvalues, eigenvectors
```

Use case: Linear regression, PCA (dimensionality reduction), neural network forward-pass sab matrix multiplication (dot product) par based hote hain.

12. Random Module

```
np.random.seed(42)           # reproducibility ke liye
np.random.rand(3)            # uniform [0,1)
np.random.randn(3)           # normal distribution
np.random.randint(1,100,5)    # random integers
np.random.choice([1,2,3], 2) # random sampling
```

Use case: Train/test split, model weight initialization, data augmentation, aur simulation-based experiments (Monte Carlo) mein random module bahut use hota hai. `seed()` set karne se result har baar same aata hai (reproducibility).

13. Copy vs View

ENGLISH

This concept is asked a lot in interviews. A **view** is a reference to the original data (memory is shared) — if you change the view, the original changes too. A **copy** creates a new, independent array.

HINGLISH

Ye concept interview mein bahut poocha jata hai. **View** original data ka reference hota hai (memory share hoti hai) — agar view change kiya to original bhi change ho jayega. **Copy** ek naya independent array banata hai.

```
a = np.array([1,2,3])
b = a.view()    # view -> same memory
b[0] = 100
print(a)        # [100, 2, 3] - original bhi change hua!

c = a.copy()    # copy -> alag memory
c[0] = 999
print(a)        # a change nahi hoga
```

⚠ Slicing bhi by default view return karti hai, copy nahi — isliye bug se bachne ke liye zaruri hone par `.copy()` explicitly use karo.



Part 2: Pandas Deep Dive

14. Pandas Kya Hai Aur Kyu Zaruri Hai

ENGLISH

Pandas is a library built to handle **tabular data** (like an Excel/SQL table). It is built on top of NumPy, and makes cleaning and analyzing real-world messy data (missing values, mixed types, dates) very easy.

HINGLISH

Pandas ek library hai jo **tabular data** (Excel/SQL table jaisa) ko handle karne ke liye banayi gayi hai. Ye NumPy ke upar built hai, aur real-world messy data (missing values, mixed types, dates) ko clean aur analyze karna bahut easy bana deti hai.

Use case: Kisi bhi company ka sales data, customer data, ya survey results ko load karke analyze karna, clean karna, aur report banane ke liye Pandas industry standard hai.

15. Series & DataFrame Basics

ENGLISH

A **Series** is a single column (1D labeled array), and a **DataFrame** is a combination of multiple Series (a 2D table, like an Excel sheet).

HINGLISH

Series ek single column (1D labeled array) hoti hai, aur **DataFrame** multiple Series ka combination hota hai (2D table, jaisa Excel sheet).

```
import pandas as pd
s = pd.Series([10,20,30], index=['a','b','c'])

df = pd.DataFrame({
    'Name': ['Amit','Riya','Karan'],
    'Age': [25,30,22],
    'City': ['Delhi','Mumbai','Pune']
})
df.head()      # pehli 5 rows
df.info()      # columns, dtype, non-null count
df.describe()  # statistical summary (mean, std, min, max)
```

16. Data Reading/Writing (CSV, Excel, JSON)

```
df = pd.read_csv('data.csv')
df = pd.read_excel('data.xlsx', sheet_name='Sheet1')
df = pd.read_json('data.json')

df.to_csv('output.csv', index=False)
df.to_excel('output.xlsx', index=False)
```

Use case: Real projects mein data almost hamesha CSV/Excel/database se aata hai — ye pehla step hota hai kisi bhi data analysis pipeline ka.

17. loc vs iloc vs at/iat

Method	Kaam	Example
<code>.loc[]</code>	Label/name se select karta hai	<code>df.loc[0,'Name']</code>
<code>.iloc[]</code>	Integer position se select karta hai	<code>df.iloc[0,1]</code>
<code>.at[]</code>	Single value fast access (label based)	<code>df.at[0,'Name']</code>
<code>.iat[]</code>	Single value fast access (position based)	<code>df.iat[0,1]</code>

Use case: Bade dataframe mein sirf ek specific value badalni ho to `.at` / `.iat` use karna `.loc` / `.iloc` se fast hota hai.

18. Filtering & Conditional Selection

```
df[df['Age'] > 25]                # simple condition
df[(df['Age']>25) & (df['City']=='Pune')] # multiple condition (AND)
```

```
df[df['City'].isin(['Delhi','Mumbai'])]      # isin filter
df.query('Age > 25')                        # query syntax
```

Use case: E-commerce data mein "sirf wo customers dikhao jinhone 1000 se zyada spend kiya aur Delhi se hain" jaisi business queries yahi filtering use karti hain.

19. Missing Data Handling

```
df.isnull().sum()          # har column mein kitne NaN hain
df.dropna()                # NaN wali rows hata do
df.fillna(0)               # NaN ko 0 se bharo
df.fillna(df['Age'].mean()) # mean se bharo
df.fillna(method='ffill')  # pichli value se fill (forward fill)
```

Use case: Real-world survey data mein log kai fields blank chhod dete hain — missing values ko sahi tarike se handle karna model accuracy ke liye critical hota hai.

20. GroupBy Deep Dive

ENGLISH
GroupBy follows the "Split-Apply-Combine" pattern: split the data into groups, apply a function on each group, then combine the results.

HINGLISH
GroupBy "Split-Apply-Combine" pattern follow karta hai: data ko groups mein split karo, har group par function apply karo, phir result combine karo.

```
df.groupby('City')['Age'].mean()          # city-wise average age
df.groupby('City').agg({'Age':'mean','Salary':'sum'})
df.groupby(['City','Gender']).size()      # multiple column grouping
```

Use case: "Har region ki total sales" ya "har department ka average salary" jaise business reports GroupBy se seconds mein ban jate hain.

21. Merge, Join & Concat

```
pd.merge(df1, df2, on='ID', how='inner')  # SQL jaisa join
pd.merge(df1, df2, on='ID', how='left')   # left join
pd.concat([df1, df2], axis=0)              # rows mein jodna
pd.concat([df1, df2], axis=1)              # columns mein jodna
df1.join(df2, on='ID')                     # index-based join
```

Use case: Do alag tables ko combine karna — jaise "Customer" table ko "Orders" table se ID ke basis par jodna, exactly SQL JOIN jaisa.

22. Pivot Table & Crosstab

```
df.pivot_table(values='Sales', index='City', columns='Month', aggfunc='sum')
pd.crosstab(df['City'], df['Gender'])
```

Use case: Excel ke Pivot Table jaisa hi — sales report banate waqt "har city ka har month ka total sales" ek table mein dikhana.

23. Apply, Map & Applymap

Function	Kaam karta hai
<code>.apply()</code>	Row/column par custom function apply (Series ya DataFrame dono par)
<code>.map()</code>	Sirf Series par, element-wise mapping/transformation
<code>.applymap()</code>	Poore DataFrame ke har cell par function apply

```
df['Age_group'] = df['Age'].apply(lambda x: 'Adult' if x>=18 else 'Minor')
df['City'] = df['City'].map({'Delhi':'DEL','Mumbai':'BOM'})
```

Use case: Categorical column banana (jaise Age ko "Young/Adult/Senior" groups mein convert karna) ya custom business logic apply karna.

24. Sorting & Ranking

```
df.sort_values('Age')                # ascending
df.sort_values('Age', ascending=False)
df.sort_values(['City','Age'])        # multiple columns
df['Rank'] = df['Salary'].rank(ascending=False)
```

25. Duplicate Handling

```
df.duplicated().sum()                # kitni duplicate rows hain
df.drop_duplicates()                  # duplicates hatao
df.drop_duplicates(subset=['Name'])   # specific column ke basis par
```

Use case: Data collection ke time same entry multiple baar aa jaati hai (jaise form submit 2 baar ho gaya) — usse remove karna data cleaning ka essential step hai.

26. DateTime Handling

```
df['Date'] = pd.to_datetime(df['Date'])
df['Year'] = df['Date'].dt.year
df['Month'] = df['Date'].dt.month
df['Day'] = df['Date'].dt.day_name()
df.set_index('Date').resample('M').sum()  # monthly aggregation
```

Use case: Time-series analysis mein (jaise stock prices, sales trends) date ko proper format mein convert karke month/year-wise trend nikalna common task hai.

🎯 Part 3: Frequently Asked Interview Questions

27. NumPy Interview Questions

Q1. NumPy array aur Python list mein kya difference hai?

NumPy array homogeneous (same data type) hota hai aur memory mein contiguous block mein store hota hai, jisse operations fast hote hain. Python list heterogeneous ho sakti hai aur har element ek separate object hota hai, jo slow hota hai.

Q2. Broadcasting kya hai aur iske rules kya hain?

Broadcasting NumPy ko different shapes ke arrays par operation karne deta hai. Rule: dimensions ya to equal honi chahiye, ya unmein se ek 1 honi chahiye (trailing dimensions se compare hota hai, right se left).

Q3. axis=0 aur axis=1 mein kya difference hai?

axis=0 operation ko rows ke through (column-wise result) apply karta hai, aur axis=1 operation ko columns ke through (row-wise result) apply karta hai.

Q4. reshape() aur resize() mein kya difference hai?

reshape() ek naya view return karta hai (original array change nahi hoti, total elements same rehne chahiye), jabki resize() original array ko in-place modify karta hai aur elements kam/zyada bhi ho sakte hain (missing values 0 se fill ho jaate hain).

Q5. copy() aur view() mein kya difference hai?

view() memory share karta hai — ek change dusre ko affect karta hai. copy() ek independent naya array banata hai, jisme changes original ko affect nahi karte.

Q6. Vectorization se performance kaise improve hoti hai?

Vectorized operations NumPy ke internal C implementation use karte hain, jo Python-level loops (jisme har iteration mein interpreter overhead hota hai) se kaafi zyada fast hote hain.

Q7. np.nan aur None mein kya difference hai?

np.nan ek float value hai jo "Not a Number" represent karta hai aur numeric arrays mein use hota hai. None Python ka generic missing value object hai jo object dtype arrays mein use hota hai.

28. Pandas Interview Questions

Q1. Series aur DataFrame mein kya difference hai?

Series ek single-dimensional labeled array hai (ek column). DataFrame multiple Series ka collection hai — 2D tabular structure with rows aur columns.

Q2. loc aur iloc mein kya difference hai?

loc label/index-name based selection karta hai (jaise df.loc['a']), jabki iloc integer position based selection karta hai (jaise df.iloc[0]) — chahe index labels kuch bhi hon.

Q3. merge() aur concat() mein kya difference hai?

merge() SQL-style join karta hai kisi common key/column ke basis par (inner, left, right, outer). concat() simply do ya zyada dataframes ko rows ya columns ke basis par ek dusre ke saath jod deta hai, bina kisi key ke.

Q4. apply(), map() aur applymap() mein kya difference hai?

map() sirf Series par element-wise kaam karta hai. apply() Series aur DataFrame dono par (row/column level) kaam karta hai. applymap() poore DataFrame ke har individual cell par function apply karta hai.

Q5. Missing values handle karne ke tarike kya hain?

dropna() se missing rows/columns hata sakte hain, fillna() se koi specific value, mean/median, ya forward/backward fill se bhar sakte hain, aur interpolate() se estimated values se fill kar sakte hain.

Q6. GroupBy internally kaise kaam karta hai?

GroupBy "Split-Apply-Combine" strategy follow karta hai: pehle data ko groups mein split karta hai (based on column values), phir har group par ek aggregation/transformation function apply karta hai, aur last mein results ko wapas ek dataframe mein combine karta hai.

Q7. pivot_table() aur groupby() mein kya difference hai?

groupby() flexible hai aur multiple aggregations ke liye use hota hai lekin output long-format mein aata hai. pivot_table() specifically ek reshape operation hai jo Excel jaisi cross-tabulated (wide-format) summary table banata hai.

Q8. inplace=True parameter kya karta hai?

inplace=True use karne se operation original dataframe ko directly modify kar deta hai (memory mein), aur function None return karta hai. Agar inplace=False (default) hai, to ek naya dataframe return hota hai aur original unchanged rehta hai.

Q9. Pandas mein memory usage kaise optimize karte hain bade datasets ke liye?

Data types ko optimize karke (jaise int64 ki jagah int32/int8, object ki jagah category dtype use karna), chunksize parameter se CSV ko chunks mein read karke, aur unnecessary columns ko drop karke memory usage kaafi kam kiya ja sakta hai.

Q10. duplicated() aur drop_duplicates() mein kya farak hai?

duplicated() ek Boolean Series return karta hai jismein batata hai kaunsi row duplicate hai (True/False). drop_duplicates() actually un duplicate rows ko dataframe se remove kar deta hai.